Combating Abusive Information in Encrypted Messaging Services: A Secure and Efficient Realization

Rui Lian, Yifeng Zheng, *Member, IEEE*, Yulong Ming, Chengjun Cai, *Member, IEEE*, Cong Wang, *Fellow, IEEE*, and Xiaohua Jia, *Fellow, IEEE*

***Abstract*—End-to-end encrypted messaging services (EEMSs) empower private communication through encrypting messages, yet also make content moderation for combating the spread of abusive messages challenging. There is an urgent call for supporting content moderation in EEMSs while ensuring user privacy. In this paper, we present a new system design for privacy-assured content moderation in EEMSs. At a high level, users in our system can privately report abusive messages, and the EEMS traces the source if a message has an aggregated report count exceeding a predefined threshold and is audited to be abusive. Our system mainly departs from prior works in that it allows flexible and adaptable thresholds, offers robustness against dishonest reporters providing malformed reports, and better ensures the privacy of all users during the moderation process. We also take a step further and propose a privacy-aware detection mechanism that relies on a blocklist built with transparency to mitigate the further spread of identified abusive messages from forwarders. Formal security analysis is provided and extensive experiments demonstrate the practical efficiency of our system.**

***Index Terms*—Content moderation, end-to-end encryption, threshold aggregation.**

I. Introduction

TODAY'S widely used end-to-end encrypted messaging services (EEMSs), such as WhatsApp, Signal, Telegram, and Facebook Messenger, employ end-to-end encryption [2] to conceal plaintext messages, preventing unauthorized access by any entities other than the intended recipients. While the privacy-preserving feature offered by EEMSs is intriguing, it is also subject to exploitation by some users to spread abusive information (such as fake news, disinformation, and child sexual abuse material [3], [4]) for varying purposes. Such misuse of EEMSs may lead to highly detrimental social consequences [5], [6], [7], and has received increasing attention in recent years [8], [9]. There is thus an urgent call to investigate how to support content moderation without compromising user privacy in EEMSs.

There has been a line of work towards privacy-preserving content moderation in EEMSs, which in general can be classified into two categories: blocklist-based and user-reporting-based. Blocklist-based approaches [10], [11], [12], [13], [14] mainly rely on using a pre-defined blocklist, which is built from known-bad content, to scan images based on encrypted similarity matching protocols. A pre-defined blocklist, however, may fall short of capturing newly emerging harmful content in a timely manner, which would limit its effectiveness in practical deployments where new abusive messages keep arising over time. In contrast, user-reporting-based methods [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25] mainly relies on users for reporting abusive messages with various privacy enhancements, where the EEMS tracks down the relevant users (senders, forwarders, or sources) of abusive messages once they are identified. It is noted that simply relying on a single report [19], [20], [21], [22] to trigger source tracing can lead to burdensome moderation workloads for EEMSs in practice. A more practical strategy is to adopt a threshold-based strategy [10], where the EEMS sets a threshold and moderates messages once their report counts reach that threshold. Considering that abusive messages may often spread rapidly, it is also necessary to adjust the threshold over time. For instance, Meta has been lowering the moderation threshold in response to a rise in hateful content on Instagram [26]. While there have been some privacy-aware user-reporting protocols [23], [24] under the threshold-based setting, they are not yet satisfactory for use due to the following downsides. First, prior works lack support for flexible threshold settings. The work in [23] fails to support lowering the threshold below 100 for accurate counting due to the use of probabilistic data structures. Bell et al. [24] reduce many operations executed between the two servers, achieving good efficiency. However, it requires time and space complexity $\mathcal{O}(N \cdot t)$ for prove and verification, where N is the number of reports and t is the threshold. This means the increase of the threshold will increase the computation time and communication cost for prove and

Received 12 February 2025; revised 19 June 2025; accepted 29 September 2025. Date of publication 1 October 2025; date of current version 11 December 2025. This work was supported in part by the National Cryptologic Science Fund of China under Grant 2025NCSF02033, in part by the Hong Kong Research Grants Council under Grant CityU 11218322, Grant 11219524, Grant R6021-20F, Grant R1012-21, Grant RFS21221S04, Grant C2004-21G, Grant C1029-22G, Grant C6015-23G, and Grant N_CityU139/21, and in part by the Innovation and Technology Commission (ITC) under the Joint Mainland-Hong Kong Funding Scheme (MHKJFS) under Grant MHP/135/23. in part by the InnoHK initiative, The Government of the HKSAR, and the Laboratory for AI-Powered Financial Technologies (AIFT). An earlier version of this paper was presented at the European Symposium on Research in Computer Security (ESORICS), 2024 [DOI:10.1007/978-3-031-70890-9_13]. *(Corresponding author: Cong Wang.)*

Rui Lian, Yulong Ming, Cong Wang, and Xiaohua Jia are with the Department of Computer Science, City University of Hong Kong, Hong Kong (e-mail: rui.lian@my.cityu.edu.hk; myl.7@my.cityu.edu.hk; congwang@cityu.edu.hk; csjia@cityu.edu.hk).

Yifeng Zheng is with the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, Hong Kong, and also with The Hong Kong Polytechnic University Shenzhen Research Institute, Shenzhen 518057, China (e-mail: yifeng.zheng@polyu.edu.hk).

Chengjun Cai is with the Department of Computer Science, City University of Hong Kong (Dongguang), Dongguang 523808, China (e-mail: chengjun.cai@cityu-dg.edu.cn).

Digital Object Identifier 10.1109/TSC.2025.3616516

1939-1374 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

verification. Second, prior works overlook the realistic threat that there could be dishonest users exploiting the privacy-preserving reporting process to submit malformed reports (e.g., a report with an enormous count or a fabricated report of a non-received message) to disrupt the moderation service. Third, prior works fail to provide strong user privacy protection, as the EEMS would be able to learn the links between a reported message and reporters, receivers, and/or forwarders through the source tracing process. This may discourage users from actively participating in reporting abusive information in EEMSs.

In light of the above, in this paper, we present a new privacy-assured content moderation system for EEMSs with promising performance. Similar to prior works [23], [24], our system also adopts a user-reporting-based strategy under a threshold-based setting, where users can report abusive messages yet with their privacy protected. We develop a customized protocol to enable private counting of user reports, where individual reports are strictly kept confidential. The protocol also supports secure threshold evaluation for aggregated report counts, which is efficient and does not pose constraints on the concrete threshold value. It is also designed with robustness against dishonest reporters attempting to provide malformed reports.

In designing this protocol, our starting point is to employ distributed point function (DPF) [27] to hide the connection between reporters and their reported messages while allowing correct aggregation of reports from different users. We then further incorporate the techniques of secret-shared shuffle [28] and parity segment trees [29] to allow oblivious and efficient evaluation against a pre-defined threshold (which can be flexibly set). As for robustness against malformed reports, we devise a MAC-based proof construction to validate DPF-based reports by combining information-theoretic MACs with DPF schemes, ensuring a reporter can increment the report count for a received message only by one. On top of the privacy-preserving design for reporting and tracing back abusive messages, we take a step further and propose a privacy-aware design to mitigate the spread of abusive messages from forwarders once they are identified. The goal is to ensure that the message for detection and the detection result are hidden from the EEMS and only revealed to the forwarder, so that forwarders are cautioned before they proceed with forwarding. For this goal, one might attempt to build a blocklist using identified abusive messages and integrate private blocklist-based detection solutions [10], [11], [12], [13], [14] to detect forwarders' messages. However, there are two issues here. First, a blocklist that includes only abusive messages audited by the EEMS lacks transparency. Second, existing approaches focus on image data and also suffer from the problem of false positives. So they are not applicable in our system. We propose a scheme based on the threshold signature technique [30] for building a transparent blocklist for detection where no single entity can add any item to the blocklist independently. We integrate it with DPF to foster a transparent and oblivious detection service, where only the forwarder can learn the detection result and obtain the aggregate signature of the matched item for verification.

We provide formal security analysis of our system, and make a GPU-accelerated prototype implementation, on top of which extensive experiments are conducted for performance evaluation. The evaluation results demonstrate the practical usability of our system. For example, our system only takes 0.242 s to privately check if the report counts of 10^5 messages exceed a threshold and 32.638 ms to obliviously response to a detection over a blocklist holding 10^5 items.

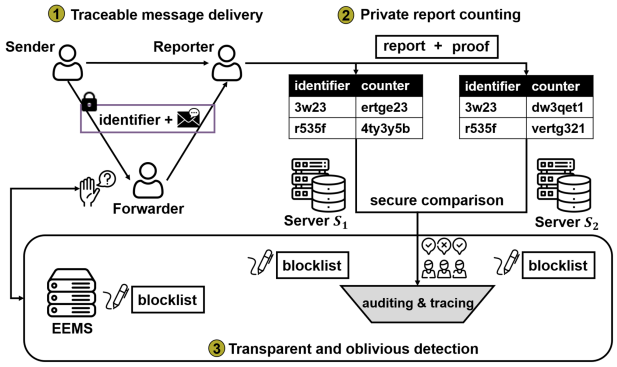


Fig. 1. The system architecture.

The remainder of the paper is structured as follows. Section II gives an overview of our system. Section III introduces some preliminaries. Section IV elaborates on the technical design of our system. Section V presents formal security definitions and proofs. Section VI provides the implementation and experiment results. Section VII discusses the related work. Section VIII concludes the whole paper.

II. System Overview

A. Architecture

Fig. 1 shows the architecture of our moderation system designed to mitigate the spread of abusive messages in EEMSs, which enables private threshold-enabled reporting-then-tracing service and transparent blocklist-driven detection service for abusive messages in an oblivious manner. The system comprises three entities: the EEMS, users, and a private moderation service provider (PMS). Users deliver messages to intended users via the EEMS's end-to-end encryption channel. Our system supports reporting-then-tracing for the messages in EEMS, and thus a custom identifier tuple is also delivered alongside the message to the intended recipient, aligning with prior traceback systems for EEMSs [19], [20], [22], [23], [24].

When a user deems the received message abusive, the associated identifier tuple is encoded as a report submitted to the PMS. To prevent a malformed report from disrupting the private report counting, the reporter is required to submit proof for the report. Upon receiving reports with proofs, the PMS first blindly verifies the validity of the reports and updates the report counters of messages without learning which messages are being reported by the users. This process continues until the end of a predefined period (e.g., the epoch). Then, the PMS privately compares the report counts of the messages against a threshold, which can be flexibly defined and adjusted by the EEMS. Messages' report counts exceeding this threshold are flagged, and their identifier tuples are returned to the EEMS for auditing whether they are abusive and tracing the sources of abusive messages.

To further mitigate the spread of abusive messages in EEMS, our system alerts a forwarder against a message's toxicity before forwarding it, which could be helpful as studied in the works [12], [31]. Specifically, upon flagging messages' identifier tuples from the above process, the PMS performs transparent and oblivious detection for forwarded messages based on a blocklist. To build a transparent blocklist, the EEMS and the PMS exclusively review the associated messages' plaintext. If a message is audited as abusive by both, the EEMS traces the message source. The related identifier tuple is signed by the auditing

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

parties and added to the continuously updated blocklist (held by both), where no single party can generate a signature to add an item to the blocklist. When a user wants to forward a message, they encode its identifier tuple and send the encoded data to the auditing parties. The parties perform oblivious detection over the blocklist and return the concealed detection result to the user without learning anything about the forwarded message. Based on this result, the user decides whether to forward the message.

B. Threat Model

We model the underlying EEMS as a closed box, abstracting its internal workings while following standard threat models for EEMSs in the literature [2], [32], [33]. The EEMS reveals who communicates with whom and maintains forwarding capabilities, adhering to standards employed by platforms such as WhatsApp, Signal, and various academic proposals [19], [20], [22], [23]. The EEMS operates semi-honestly, faithfully executing the tasks assigned within the system.

For users, we consider a variety of potential actions. During message delivery, a dishonest source may attempt to send a message that evades tracing by forging its origin to appear as though it was sent by an innocent user. When reporting messages, a dishonest reporter may manipulate the private report tally by submitting false information, such as adding an integer greater than one or a negative integer to the report count of a message, thereby influencing the speed of a message being flagged and audited. In the forwarding context, we assume the forwarder follows the prescribed protocol since the detection goal is to warn the forwarder of potential harm without revealing the message in plaintext. These assumptions align with those commonly adopted in prior studies on moderation for EEMSs [10], [11], [12], [13], [14], [19], [20], [21], [22], [23], [24].

For the PMS, we assume the presence of two non-colluding servers, S_1 and S_2, located in distinct trust domains (which could be taken up by independent content moderation platforms like Hive Moderation, Amazon Rekognition, Alibaba Cloud in practice), as is common in trending practices [34], [35], [36]. The two servers are assumed to be semi-honest, meaning that they will follow the execution protocols correctly but may independently attempt to infer sensitive information, such as the report counts or the content of forwarded messages, due to various potential interests.

We assume each participant is assigned unique identity keys when registered in the system, which can be verified through out-of-band channels if necessary. Side-channel and Sybil attacks are outside the scope of this paper.

C. Security Objectives

We aim to achieve the following security objectives:

- *Message confidentiality.* Both original and forwarded messages are inaccessible to non-receiving users, the EEMS, and the PMS. The messages whose report counts reach the threshold are audited by the EEMS and PMS to determine whether they are added to a blocklist.
- *Source traceability.* Users cannot send messages without the possibility of tracing them back to their identity, nor can they falsely accuse another user of sending a message they did not originate.
- *Privacy and robustness in report counting.* Messages' report counts are concealed against all parties, while the threshold is predefined by the EEMS and can be modified without a system reset or extra costs. Reporters cannot falsely report on messages they haven't received or misreport received messages with incorrect values.
- *Transparency and obliviousness in blocklist-based detection.* No single party can add a message to the blocklist. A forwarder only learns the detection result without knowing the blocklist content, except for the entry that matches the forwarded message.

Algorithm 1: Secure Identifier Generation (SIdGen).

Input: U_s holds m and the EEMS holds sk_E.
Output: U_s holds $\mathcal{I}$ and the EEMS holds the key-value pair $\{\mathsf{H}(c) : (c, ik)\}$.

Source U_s: ▷ Generate items of an identifier tuple
1: $mk \leftarrow \mathsf{keyGen}_{sym}(1^\lambda)$;
2: $c \leftarrow \mathsf{E}_{mk}(m)$; $ch \leftarrow \mathsf{E}_{mk}(\mathsf{H}(c))$;
3: deliver (c, ch) to the EEMS;
The EEMS: ▷ Generate items of an identifier tuple
4: $ik \leftarrow \mathsf{keyGen}_{sym}(1^\lambda)$;
5: $src \leftarrow \mathsf{E}_{ik}(U_s)$; $\sigma \leftarrow \mathsf{Sign}_{sk_E}(src, ch)$;
6: store key-value pair $\{\mathsf{H}(c) : (c, ik)\}$;
7: deliver (σ, src) to U_s;
Source U_s: ▷ Create the identifier tuple
8: $\mathcal{I} = (mk, src, \mathsf{H}(c), \sigma)$;

III. Preliminaries

Function secret sharing (FSS). Two-party Function Secret Sharing (FSS) [27], [37] decomposes a function f into two compact shares. Each share reveals no information about the function f, and only the sum of the evaluation outputs at a given index x results in the corresponding value of $f(x)$. Formally, a two-party FSS scheme consists of a pair of algorithms $(\mathsf{FSS.Gen}, \mathsf{FSS.Eval})$: (1) $\mathsf{FSS.Gen}(1^\lambda, f) \rightarrow (k_1, k_2)$: Given a security parameter λ and a function description f, generate two function shares k_1 and k_2 that define two function keys for f, (2) $\mathsf{FSS.Eval}(k_b, x) \rightarrow y_b$: Given an input x and the function share k_b, output y_b, which corresponds to party b's share of $f(x)$.

Information-theoretic MAC. An information-theoretic Message Authentication Code (MAC) based on SPD$\mathbb{Z}_{2^k}$ [38] is defined as follows: For a secret value $\beta \in \mathbb{Z}_{2^n}$ and a key $mk \in \mathbb{Z}_{2^s}$, the MAC is given by $\mathsf{MAC}_{(mk)}(\beta) = mk \cdot \beta \in \mathbb{Z}_{2^{n+s}}$. This MAC is linear and additively homomorphic, i.e., $mk \cdot \beta = mk \cdot [\beta]_1 + mk \cdot [\beta]_2$ where $\beta = [\beta]_1 + [\beta]_2$. The probability that an adversary can forge a valid MAC is $1/2^s$.

Additive secret sharing (ASS). Let $\mathbb{Z}_{2^n}$ denote the ring of integers modulo 2^n. Given a secret value $x \in \mathbb{Z}_{2^n}$, an additive secret sharing (ASS) scheme [39] generates two additive shares $[x]_1$ and $[x]_2$ such that $x = [x]_1 + [x]_2$. Each share is uniformly distributed in the ring, ensuring that no information about x is revealed from either share alone.

IV. Detailed Design

In this section, we first present an overview of the design and then show the design rationale behind it, which we use to implement these phases discussed and provide concrete descriptions of the associated algorithms.

Design overview. Since messages are continuously generated and delivered, our system operates in epochs like [23], [24], where each epoch includes M messages and a set $\mathcal{N}$ of reporters

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

and each reporter $U_i \in \mathcal{N}$ issues a report for a message and the EEMS holds a threshold t. Each epoch consists of three distinct phases, i.e., (1) traceable message delivery (Section IV-A), (2) private report counting (Section IV-B), and (3) transparent and oblivious detection (Section IV-C). To secure these phases, we identify the necessary algorithms as follows: SIdGen, SIdVrf, PrRprE, BPrfGen, BPrfVrf, RptTally, STrsEval, and TODtc, and build on secure cryptographic techniques to appropriately realize these algorithms. These designed algorithms are decoupled from the end-to-end encryption algorithms used by an EEMS. Thus, each EEMS can create an instance in the system and work independently.

In phase (1), a source produces a custom identifier tuple $\mathcal{I}$ by calling SIdGen. The $\mathcal{I}$ is delivered with the message m, which is verified by the receiver by calling SIdVrf to make sure that $\mathcal{I}$ corresponds to m and the source identity has not been modified.[1] In phase (2), $S_{\{1,2\}}$ of the PMS carry out private report counting service. Each reporter issues a report and proof pair by running PrRprE and BPrfGen and delivers the pair to $S_{\{1,2\}}$, which run BPrfVrf to verify the pair. Note that each pair is encoded from $\mathcal{I}$ of the reported message from $\mathcal{N}$.[2] They then perform report-hiding tally to update the counters for the M messages by calling RptTally and secure threshold evaluation that compares the counts with the threshold t (which can be flexibly defined by the EEMS and is easily adaptable) by calling STrsEval and returns the flagged identifier tuples $\ddot{\mathcal{I}}$ of the corresponding messages (i.e., the ones that hit t) to the EEMS. In phase (3), Given $\ddot{\mathcal{I}}$, the EEMS and $S_{\{1,2\}}$ build a transparent blocklist where each item includes the $\mathcal{I}$ and an aggregate signature σ^* of audited abusive message. The EEMS recovers the sources of audited abusive messages. Then, they execute oblivious detection by running TODtc, which uses inputs from a forwarder over the blocklist, which returns the masked detection result to the forwarder.

A. Traceable Message Delivery

Design rationale. To facilitate source traceability of messages within the EEMS, a common approach is to construct an identifier tuple that associates the message with the source identity and subsequently attach this identifier tuple to the message as it is transmitted through the EEMS's end-to-end encrypted channel. Prior identifier tuple designs [20], [21], [22], [23] require to be disclosed alongside the plaintext message by a reporter, thereby enabling a verifiable report for the message to the EEMS. Such a combination disclosure is not straightforward to integrate into our threshold-based reporting-then-tracing system. Revealing the reported message would undermine the reporter's privacy and contradict the goal of private report counting. Conversely, we adopt the fundamental structure of an identifier tuple and modify it to operate verifiably and privately within our system. We devise a tailored identifier tuple $\mathcal{I}$, which is produced by the originator with the support of the EEMS, aligning with the methodologies established in previous studies [20], [21], [22], [23]. We acknowledge the possibility of a malevolent source presenting a counterfeit $\mathcal{I}$, which may either fail to accurately associate a message with its true source identity or may pertain to a non-existent message. Adhering to the trust paradigms of preceding works [20], [21], [22], [23], we delegate the duty of validating $\mathcal{I}$ to the recipient parties. An $\mathcal{I}$ is also obtained by $S_{\{1,2\}}$ to derive a unique index for private report counting, which is conveyed back to the EEMS if the report count for the associated message reaches a predetermined threshold (further elaborated in Section IV-B). A forwarder encodes $\mathcal{I}$ of the message being forwarded and transmits the encoded data to $S_{\{1,2\}}$ for obliviously detecting whether the forwarded message is abusive, as detailed in Section IV-C. In brief, we develop Secure Identifier Generation (SIdGen) and Secure Identifier Verification (SIdVrf) algorithms where SIdGen is used by a source to produce an $\mathcal{I}$ while SIdVrf is employed by the recipient to ascertain the validity of $\mathcal{I}$.

Algorithm 2: Secure Identifier Verification (SIdVrf).

Input: U_r holds m and pk_E while $S_{\{1,2\}}$ hold pk_E.
Output: The verification result is 1 or 0.

```
get mk, src, H(c), σ from I;
execute if called by U_r
c' ← E_mk(m);
if H(c) ≠ H(c') then Return 0;
end if
execute whoever call
ch' ← E_mk(H(c));
Return Verify(pk_E, σ, src, ch');
```

Construction. We commence by defining several notations. We use λ to denote a security parameter and $\perp$ to represent an unsuccessful operation. We employ a symmetric encryption scheme defined by the tuple $(k \leftarrow \mathsf{keyGen}_{sym}(1^\lambda), c \leftarrow \mathsf{E}_k(m), m \leftarrow \mathsf{D}_k(c))$, a signature scheme given by the tuple $((pk, sk) \leftarrow \mathsf{keyGen}_{sig}(1^\lambda), \sigma \leftarrow \mathsf{Sign}_{sk}(m), \{0,1\} \leftarrow \mathsf{Verify}(pk, \sigma, m))$, and a collision-resistant hash function $\mathsf{H} : \{0,1\}^* \rightarrow \mathbb{Z}_{2^{n+s}}$. We assume that the EEMS possesses a long-term public key pk_E and private key sk_E. Furthermore, each user is allocated a unique identifier string U_i, which is authenticated by the system for user identification. Let us now consider a scenario where a source U_s possesses a new plaintext message m. We illustrate the process of generating the corresponding identifier tuple $\mathcal{I}$ by invoking SIdGen and verifying it by employing SIdVrf, as detailed in Algorithm 1 and Algorithm 2. We adopt a design where the EEMS signs both the message and its source identity while preserving confidentiality. First, U_s encrypts m into the ciphertext c and computes its encryption ch of hash $\mathsf{H}(c)$ using a random encryption key mk, delivering c and ch to the EEMS. The EEMS then encrypts the source identity $src \leftarrow \mathsf{E}_{ik}(U_s)$ (where ik is a key known only to the EEMS), executes $\sigma \leftarrow \mathsf{Sign}_{sk_E}(src, ch)$, and returns σ to U_s. The EEMS also stores the mapping $\{\mathsf{H}(c) : (c, ik)\}$. The identifier $\mathcal{I}$ is composed of $\{mk, src, \mathsf{H}(c), \sigma\}$ Unlike prior commitment-based schemes [20], [23] that lack plaintext recovery, our approach enables the EEMS to recover the message for auditing and allows $S_{\{1,2\}}$ to verify without compromising confidentiality. Instead of using the ciphertext c directly as an item of the tuple (which could expose m), we introduce ch to bind c and mk for secure verification. To verify $\mathcal{I}$, a receiver U_r checks if σ and $\mathsf{H}(c)$ match the plaintext message m, encryption key mk, and encrypted source identity src. Similarly, $S_{\{1,2\}}$ verify σ using mk, src, and $\mathsf{H}(c)$, ensuring message confidentiality and source traceability.

[1] The verification would not reveal the source identity to a receiver in a forward state.

[2] We limit the number of reports per reporter per epoch to avoid a reporter overwhelming the system.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: Private Proof Generation (PPrfGen).

Input: U_i holds $\mathcal{I}$.
Output: $S_{\{1,2\}}$ hold proof $[\tau]_1, [\tau]_2$ for DPF keys k_1^i, k_2^i.
1: Get mk from $\mathcal{I}$;
2: $\gamma \xleftarrow{R} \mathbb{Z}_{n+s}$; $\tau \leftarrow \mathcal{I}.mk_i \cdot 1 + \gamma \bmod 2^{n+s}$;
3: sample $[\tau]_1, [\tau]_2 \xleftarrow{R} \mathbb{Z}_{n+s}$ such that $\tau = [\tau]_1 + [\tau]_2$;
4: deliver $(\gamma, [\tau]_1)$ to S_1 and $(\gamma, [\tau]_2)$ to S_2;

Algorithm 4: Private Proof Verification (PPrfVrf).

Input: $S_{\{1,2\}}$ hold the indexes $\boldsymbol{idx}$, shuffled identifier matrix $\tilde{\boldsymbol{\mathcal{I}}}^M$, the DPF keys k_1^i, k_2^i, and proof $[\tau]_1, [\tau]_2, \gamma$.
Output: The verification result is 1 or 0.
1: S_b initiate tp_b; $\triangleright\ b \in \{1, 2\}$;
2: **for** $x \in \boldsymbol{idx}, \mathcal{I} \in \tilde{\boldsymbol{\mathcal{I}}}^M$ **do**
3: S_b get $mk_x \leftarrow \mathcal{I}.mk$;
4: S_b compute $tp_b \leftarrow tp_b + mk_x \cdot \mathsf{DPF.Eval}(k_b^i, x)$;
5: **end for**
6: S_b compute $[\omega]_b \leftarrow [\tau]_b - tp_b$;
7: S_1 and S_2 exchanges $[\omega]_1$ and $[\omega]_2$;
8: **if** $\gamma = [\omega]_1 + [\omega]_2$ **then Return** 1;
9: **else if** $\gamma \neq [\omega]_1 + [\omega]_2$ **then Return** 0;
10: **end if**

B. Private Report Counting

We proceed to detail the process by which a reporter creates an encoded report accompanied by proof (as detailed in Section IV-B1), which facilitates report-hiding tally (covered in Section IV-B2) and secure threshold evaluation (explored in Section IV-B3) carried out by the PMS ($S_{\{1,2\}}$).

1) Generating Encoded Report With Proof: Private report encoding. We follow the trending efficient two-party distributed point functions (DPF) [40], [41] for private report encoding and aggregation. DPF is a special FSS that guarantees $\mathsf{DPF.Eval}(k_1, x) + \mathsf{DPF.Eval}(k_2, x) = \beta$ only if $x = \alpha$ for $k_1, k_2 \leftarrow \mathsf{DPF.Gen}(\alpha, \beta)$ where $\alpha, \beta \in \mathbb{Z}_{2^n}$ and $k_1, k_2 \in \mathbb{Z}_{2^p}$. Our idea is to let a reporter U_i produce a pair of DPF keys k_1^i, k_2^i for a point function parameterized by the identifier tuple $\mathcal{I}$[3] of the reported message and value one. k_1^i, k_2^i are submitted to $S_{\{1,2\}}$. For the DPF keys from $\mathcal{N}$, $S_{\{1,2\}}$ can compute secret shares $[y^x]_1, [y^x]_2$ of the report count $y^x \in \mathbb{Z}_{2^n}$ for the message with index $x \in \mathbb{Z}_{2^n}$ by evaluating all DPF keys from $\mathcal{N}$ on the index $x \in \mathbb{Z}_{2^n}$. Officially, with a cryptographic hash function $\tilde{\mathsf{H}} : \{0,1\}^* \to \mathbb{Z}_{2^n}, n < k$, Private Report Encoding (PrRprE) algorithm is defined as: $(k_1^i, k_2^i) \leftarrow \mathsf{DPF.Gen}(\alpha = \tilde{\mathsf{H}}(\mathcal{I}), \beta = 1)$.

Lightweight proof generation. To counteract dishonest reporters who might falsely report unreceived messages by arbitrarily setting α or manipulate the threshold reach for received messages by choosing $\beta \neq 1$ in the point function $f_{\alpha,\beta}$, $S_{\{1,2\}}$ are supposed to verify the DPF keys submitted by reporters. Previous works [40], [41] have introduced methods to defend against malformed DPF keys that could alter the aggregate evaluation outcome by setting $\beta \neq 1$. Boyle et al. [40] utilize a random matrix $\mathcal{L} \in \mathbb{Z}_{2^n}^3$, specifically $L_1 = \boldsymbol{r}, L_2 = \boldsymbol{s}, L_3 = \boldsymbol{r} \cdot \boldsymbol{s}$, where $\boldsymbol{r}$ and $\boldsymbol{s}$ are random vectors in $\mathbb{Z}_{2^n}$, to validate the pair of DPF keys k_1^i, k_2^i. For any $x \in \mathbb{Z}_{2^n}$, $S_{\{1,2\}}$ compute $\mathsf{DPF.Eval}(k_1^i, x) \cdot \mathcal{L}$ and $\mathsf{DPF.Eval}(k_2^i, x) \cdot \mathcal{L}$ locally and engages in a reporter-assisted multiparty computation protocol (based on Beaver's circuit randomization technique [42]), accepting the DPF keys only if the protocol yields a matrix of zeros. Conversely, Boneh et al. [41] enhance the DPF construction, where each key comprises a public and private component. Duplicate public components in two DPF keys indicate they can be expanded to vector shares, with a value at one point and zero elsewhere. However, they do not support verifying whether the reporter received the message. To address this issue, we propose a solution employing information-theoretic MACs [38] to detect dishonest actions. We design Private Proof Generation (PPrfGen) and Private Proof Verification (PPrfVrf) where BPrfGen is called by a reporter to generate an efficient proof for the DPF keys and BPrfVrf is used by $S_{\{1,2\}}$ verifying proof, as depicted in Algorithm 3 and Algorithm 4.

Specifically, a reporter U_i, holding a secret key $mk_i \in \mathbb{Z}_{2^s}$ linked to the message's identifier tuple $\mathcal{I}$, computes a MAC value τ as the proof, effectively "hashing" $\beta \in \mathbb{Z}_{2^n}$ with mk_i, i.e., $\tau \leftarrow mk_i \cdot \beta \bmod \mathbb{Z}_{2^{n+s}}$. The proof and the DPF key pair k_1^i, k_2^i for the point function $f_{\alpha_i,\beta}$ are submitted to $S_{\{1,2\}}$, which locally "hash" the evaluations at positions indexed by $x \in \boldsymbol{idx}$ (with $\boldsymbol{idx}$ being the index vector held by $S_{\{1,2\}}$, detailed in Section IV-B2) using its private key mk_x, i.e., $mk_x \cdot \mathsf{DPF.Eval}(k_b^i, x)$ for $b \in \{1, 2\}$. Thanks to DPF's correctness, the sum of these evaluations equals $mk_i \cdot \beta$ if the reporter only reports a received message once. Since β is consistently one, using this method would reveal the reported message due to the unique private key mk_x associated with each identifier tuple. Drawing inspiration from [36], we introduce a random key to conceal the private key, where τ "hashes" a value with a private key and adds a random key $\gamma \in \mathbb{Z}_{2^{n+s}}$. Within that, the sum recovers γ instead of revealing the unique private key mk_x if the report is valid. Correctness holds since $[\omega]_1 + [\omega]_2 = [\tau]_1 - \sum mk_x \cdot \mathsf{DPF.Eval}(k_1^i, x) + [\tau]_2 - \sum mk_x \cdot \mathsf{DPF.Eval}(k_2^i, x) = [\tau]_1 + [\tau]_2 - (\sum mk_x \cdot \mathsf{DPF.Eval}(k_1^i, x) + \sum mk_x \cdot \mathsf{DPF.Eval}(k_2^i, x)) = [\tau]_1 + [\tau]_2 - \sum mk_x \cdot (\mathsf{DPF.Eval}(k_1^i, x) + \mathsf{DPF.Eval}(k_2^i, x)) = \tau - mk_i \cdot 1 = \gamma$.

2) Report-Hiding Tally: As described in Section IV-B1, $S_{\{1,2\}}$ are tasked with evaluating DPF keys across the entire spectrum of index $x \in \mathbb{Z}_{2^n}$. To optimize this process, we refine the evaluation to the M messages produced within a single epoch, considering that M is markedly smaller than 2^n. The central dilemma is to $S_{\{1,2\}}$ to obtain identifier tuples from sources without disclosing the link between the source and the message. Inspired by the work [28], our principal strategy involves $S_{\{1,2\}}$ receiving identifier tuples that have been rearranged according to two distinct random permutations, which are kept undisclosed to each other.

Construction. In detail, the M identifier tuples are represented as a matrix $\boldsymbol{\mathcal{I}}^M$, organized based on their chronological order of generation such that each row in the matrix corresponds to a tuple. Each source transmits secret shares of its identifier tuple to $S_{\{1,2\}}$, such that by the epoch's end, $S_{\{1,2\}}$ possess two secret shares of a matrix $\boldsymbol{\mathcal{I}}^M$. The two secret shares are represented as two matrices $[\boldsymbol{\mathcal{I}}^M]_1, [\boldsymbol{\mathcal{I}}^M]_2$. Subsequently, $S_{\{1,2\}}$ perform a secret-shared shuffle protocol [28] that uses the two secret

[3]An identifier tuple $\mathcal{I} \in \mathbb{Z}_{2^k}$ is appended to messages, transmitted from senders to receivers via the EEMS. The generation process is detailed in Section IV-A.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

shares as input to obtain a corresponding shuffled matrix $\tilde{\mathcal{I}}^M$ of $\mathcal{I}^M$, where the shuffle is performed row-wise. In particular, to perform the secret-shared shuffle protocol, the EEMS prepares input-independent correlation quantities beforehand and delivers them to $S_{\{1,2\}}$. To produce the correlation quantities, the EEMS chooses two random permutations $\pi_1, \pi_2 : \mathbb{Z}_M \to \mathbb{Z}_M$ that are applied in a row-wise manner and random matrices $\boldsymbol{a}_1, \boldsymbol{a}_2, \boldsymbol{a}_3$ with entries of size $|\mathcal{I}|$, i.e., $\boldsymbol{a}_1, \boldsymbol{a}_2, \boldsymbol{a}_3 \in \mathbb{Z}^{M\times|\mathcal{I}|}$. The EEMS computes $\Delta \leftarrow \pi_2(\pi_1(\boldsymbol{a}_3) + \boldsymbol{a}_1) - \boldsymbol{a}_2$ and we represent correlation quantities as a tuple $cor = (\pi_1, \pi_2, \boldsymbol{a}_1, \boldsymbol{a}_2, \boldsymbol{a}_3, \Delta)$, in which $\pi_1, \boldsymbol{a}_1$ are sent to S_1 and $\pi_2, \boldsymbol{a}_2, \boldsymbol{a}_3, \Delta$ are sent to S_2. Then,

1) S_2 computes and sends $z_2 \leftarrow [\mathcal{I}^M]_2 - \boldsymbol{a}_3$ to S_1.
2) S_1 computes and sends $z_1 \leftarrow \pi_1(z_2 + [\mathcal{I}^M]_1) - \boldsymbol{a}_1$ to S_2.
3) S_2 computes $\tilde{\mathcal{I}}^M \leftarrow \pi_2(z_1) + \Delta + \boldsymbol{a}_2$ and delivers it to S_1 so that $S_{\{1,2\}}$ get an identical matrix $\tilde{\mathcal{I}}^M$.

Ultimately, $S_{\{1,2\}}$ perform SIdVrf on each matrix entry to achieve a verifiable shuffled identical matrix $\tilde{\mathcal{I}}^M$, where $\tilde{\mathcal{I}}^M = \pi_2(\pi_1(\mathcal{I}^M))$. In this way, each server gains no additional knowledge about $\mathcal{I}^M$ or the correlation quantities retained by the other server. Within this optimization, $S_{\{1,2\}}$ generate a common length-M index vector $\boldsymbol{idx}$, where each index x is derived from $x \leftarrow \tilde{\mathsf{H}}(\mathcal{I})$ and $\mathcal{I}$ is an element of $\tilde{\mathcal{I}}^M$. They also establish two length-M vectors $[\boldsymbol{y}]_1$ and $[\boldsymbol{y}]_2$ to serve as report counters, with each element being the secret shares $[y^x]_1$ and $[y^x]_2$ of the report count y^x corresponding to the message with index x in $\boldsymbol{idx}$. Concurrently, $S_{\{1,2\}}$ store the DPF keys and proofs from the set $\mathcal{N}$ and execute BPrfVrf to ascertain the integrity of the reports (further details are outlined in Section IV-A). Subsequently, S_b where $b \in \{1, 2\}$ perform Report-hiding Tally (RptTally) algorithm: For every x in $\boldsymbol{idx}$, increment the corresponding counter according to $[y^x]_b = \sum_{i\in\mathcal{N}} \mathsf{DPF.Eval}(k_b^i, x)$.

3) Secure Threshold Evaluation: Upon $S_{\{1,2\}}$ have updated report counters $[\boldsymbol{y}]_1, [\boldsymbol{y}]_2$, the next step is to compare these secret counts with a private threshold $t \in \mathbb{Z}_{2^n}$ predefined by the EEMS. In this process, we have to enable the EEMS to flexibly adjust t without requiring a complete system reset or additional costs. Our solution is inspired by the works [43], [44], where $S_{\{1,2\}}$ maintain identical masked report counts through additive secret sharing [39], and then compare these with a masked threshold. To achieve identical masked report counts at $S_{\{1,2\}}$, we propose that the EEMS generates a random mask $r \in \mathbb{Z}_{2^n}$, which is then divided into additive secret shares $[r]_1$ and $[r]_2$, distributed to S_1 and S_2 respectively. Each $S_b, b \in 1, 2$, calculates $[y^x]_b + [r]_b$, and after exchanging these values, they arrive at a consistent masked report count of $y^x + r$ for the message with index x. Consequently, the challenge becomes comparing the $y^x + r$ held by $S_{\{1,2\}}$ with the masked threshold $t + r$ managed by the EEMS[4].

Basic approach. As shown in Fig. 2, our initial approach involves the utilization of distributed comparison functions (DCF) [40], [41], which are specialized FSS schemes for the comparison function $g_{\mu,\nu} : \mathbb{Z}_{2^p} \to \mathbb{Z}_{2^p}$. This function is defined such that $g(j) = \nu$ if $j > \mu$, and $g(j) = 0$ otherwise. To leverage DCF to compare the masked report counts and the masked threshold, we enable the EEMS to generate a pair of DCF keys

[4]When adding two numbers within the ring $\mathbb{Z}_{2^n}$, potential overflow can lead to erroneous comparison outcomes due to the effects of modular arithmetic. To mitigate this, we utilize shares in $\mathbb{Z}_{2^{n+s}}$ to represent numbers in $\mathbb{Z}_{2^n}$, with p set to $n + s$, where s serves as a statistical security parameter.

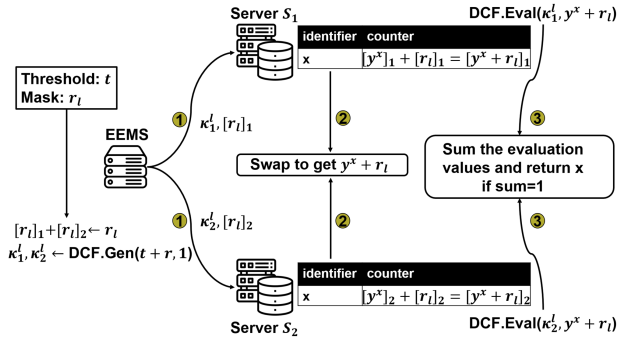


Fig. 2. An example for basic DCF-based secure threshold evaluation.

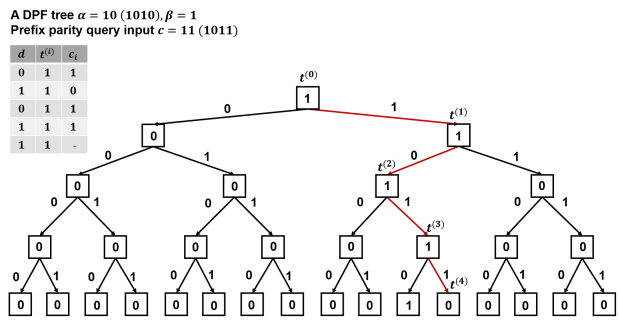


Fig. 3. An example for enhanced DPF-based prefix parity query.

corresponding to the comparison function $g_{\mu,\nu}$, with μ set to $t + r \in \mathbb{Z}_{2^{n+s}}$ and ν set to 1, and then to provide the two keys to $S_{\{1,2\}}$. They subsequently evaluate the DCF keys at $y^x + r$, exchange the outcomes of these evaluations, and verify if the aggregate equals one. If it outputs one, it indicates that the report count y^x for the message with index x surpasses the threshold t. Throughout this comparison process, neither the count nor the threshold is disclosed to $S_{\{1,2\}}$, thereby upholding the security objective of our system. Moreover, this approach facilitates adjustable threshold settings with consistent communication expenses.

Enhanced approach. We identify a new comparison method [29] that offers less communication and computation costs than our basic DCF-based comparison method. At a high level, the new method is to leverage prefix parity query based on DPF to compare $y^x + r$ and $t + r$. Prefix parity query is performed on a parity-segment tree [29], which is a data structure where the value of each node is set to the parity of its two children. It efficiently answers a parity query on a substring (which is treated as the left child of the leaf node) of a binary string. Consider a binary string $h = h_0 h_1 \cdots h_{n+s}$ and the parity-segment tree $T(h)$, a prefix parity query algorithm takes as input a binary string $g = g_0 g_1 \cdots g_{n+s}$ and outputs the prefix substring of g over $T(h)$. It is observed that the binary tree of a point function $f_{\alpha,\beta}$ is a special parity segment tree $T(\alpha)$, where the leaf nodes only have a one-bit substring. We can improve the process of DPF.Eval (more details for the pseudocode refer to the work [40]) to perform prefix parity query with input $c = y^x + r$ for a DPF-based parity segment tree $T(t + r)$ and $S_{\{1,2\}}$ swap the output to perform an $\oplus$ operation to get the result 1 if $y^x + r > t + r$, as illustrated in Fig. 3. It is noted that the outputs of $S_{\{1,2\}}$ are independently uniformly distributed concerning α. Thus, the process's security is equal to that of DPF. In particular, we follow the most steps of the prefix parity query algorithm to traverse a DPF tree, while at

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

Algorithm 5: Secure Threshold Evaluation (STrsEval).

Input: Let $\mathsf{G} : \{0,1\}^{\lambda} \rightarrow \{0,1\}^{2(\lambda+1)}$ be a pseudo-random generator. $\gamma \xleftarrow{R} \mathbb{Z}$ denote random sampling of γ from $\mathbb{Z}$. $S_{\{1,2\}}$ hold the report counters $[\boldsymbol{y}]_1, [\boldsymbol{y}]_2$. The EEMS holds the threshold t.

Output: The EEMS holds a matrix $\ddot{\boldsymbol{\mathcal{I}}}$ that includes the identifier tuples for the messages whose report counts have been more than t.

The EEMS: ▷ Offline generation of masks and DPF keys

for $l \in [M]$ **do**
  $r_l \in \mathbb{Z}_{2^n}$; $\alpha = t + r_l$; $\beta = 1$;
  sample $[r_l]_1, [r_l]_2 \xleftarrow{R} \mathbb{Z}_{2^p}$ such that $r_l = [r_l]_1 + [r_l]_2$;
  $(\kappa_1^l, \kappa_2^l) \leftarrow \mathsf{DPF.Gen}(\alpha, \beta)$; ▷ $\kappa_1^l, \kappa_2^l \in \mathbb{Z}_{2^p}$
  deliver $(\kappa_1^l, [r_l]_1)$ to the S_1 and $(\kappa_2^l, [r_l]_2)$ to the S_2;
end for

$S_b, b \in \{1,2\}$: ▷ Online secure threshold evaluation

$S_{\{1,2\}}$ initiate $\ddot{\boldsymbol{\mathcal{I}}}$;
for $x \in \boldsymbol{idx}, \mathcal{I} \in \tilde{\ddot{\boldsymbol{\mathcal{I}}}}^M, [y^x]_b \in \boldsymbol{y}_b$ **do**
  S_b compute $[y^x + r_l]_b \leftarrow [y^x]_b + [r_l]_b \bmod \mathbb{Z}_{2^p}$;
  $S_{\{1,2\}}$ swap $[y^x + r_l]_1$ and $[y^x + r_l]_2$;
  $S_{\{1,2\}}$ get $y^x + r_l \leftarrow [y^x + r_l]_1 + [y^x + r_l]_2 \bmod \mathbb{Z}_{2^p}$;
  decompose $y^x + r_l$ to a binary string $c = c_0, \ldots, c_p$;

// Improve the process of DPF.Eval

  Parse $\kappa_b^l = s^{(0)} \parallel CW^{(1)} \parallel \cdots \parallel CW^{(p+1)}$; $t^{(0)} = b$;
  initiate $out_b = 0, d = 0$; ▷ New step
  for $i \in \{1, p\}$ **do**
    Parse $CW^{(i)} = s_{CW} \parallel t_{CW}^L \parallel t_{CW}^R$;
    $te = (t^{(i-1)} \cdot [s_{CW} \parallel t_{CW}^L \parallel s_{CW} \parallel t_{CW}^R])$;
    $\eta^{(i)} \leftarrow \mathsf{G}(s^{(i-1)}) \oplus te$;
    Parse $\eta^{(i)} = s^L \parallel t^L \parallel s^R \parallel t^R \in \{0,1\}^{2(\lambda+1)}$;
    if $x_i = 0$ **then** $s^{(i)} \leftarrow s^L, t^{(i)} \leftarrow t^L$;
    else $s^{(i)} \leftarrow s^R, t^{(i)} \leftarrow t^R$;
    end if

// New steps

    if $d \neq c_i$ **then** $d = d \oplus t^{(i-1)}$; $out_b = out_b \oplus t^{(i-1)}$;
    end if
  end for
  if $d = 1$ **then** $out_b = out_b \oplus t^{(p)}$
  end if
  $S_{\{1,2\}}$ exchange out_1 and out_2;
  if $out_1 \oplus out_2 = 1$ **then** $\ddot{\boldsymbol{\mathcal{I}}}.\mathsf{add}(\mathcal{I})$;
  end if
end for
send $\ddot{\boldsymbol{\mathcal{I}}}$ to the EEMS;

the end, an additional $\oplus$ is performed for the d and $t^{(n+s)}$ if d equals 1 because the substring marks a transition in the direction from which the substring is accessed. The computation costs of such a new way of doing secure threshold evaluation for $S_{\{1,2\}}$ and the EEMS are about $1.3\times$ and $1.2\times$ faster than the basic approach where DCF is used, while the communication cost for the EEMS reduces $1.77\times$, as shown in our experiment results of Section VI-A2. The underlying reason for the enhancement is the smaller length output of the pseudo-random function in DPF than in DCF. Formally, Secure Threshold Evaluation (STrsEval) algorithm is carried out by $S_{\{1,2\}}$ and the EEMS, as shown in Algorithm 5.

C. Transparent and Oblivious Detection

Now, we show how to build a transparent blocklist upon learning the flagged identifier tuples $\ddot{\boldsymbol{\mathcal{I}}}$, and then use the blocklist to detect forwarded messages delivered via the EEMS obliviously by designing Transparent and Oblivious Detection (TODtc) algorithm, as described in Algorithm 6.

Building transparent blocklist. To avoid arbitrarily including materials outside the claimed types of content on the blocklist $\mathcal{B}$, inspired by Apple's detection of CSAM [10], the entries of $\mathcal{B}$ are agreed by at least two servers in the EEMS and $S_{\{1,2\}}$). To audit the associated messages, the EEMS uses $\mathsf{H}(c)$ in the flagged identifier tuple $\mathcal{I}$ (recall that $\mathcal{I} = \{mk, src, \mathsf{H}(c), \sigma\}$, as detailed in Section IV-A.) as an index to retrieve the stored c, ik, and delivers the associated ciphertext c to $S_{\{1,2\}}$. The three servers decrypt c using the key mk in $\mathcal{I}$ to get message plaintext m for auditing exclusively. To build a blocklist $\mathcal{B}$, a naive method is to let each server generate a signature on m they consider abusive to be accountable for the auditing result and then select these identifier tuples whose messages obtain two signatures to join $\mathcal{B}$. For the corresponding audited abusive message m, the EEMS decrypts src in $\mathcal{I}$ using the key ik to recover U_s for taking further actions. However, such a method complicates the verification of users for the blocklist since it is inevitable to enable users to learn additional information (e.g., which servers sign a message).

Instead, we identify a *t-out-of-T* threshold signature scheme [30] that enables the generation of signature shares on a message using shared signature keys distributed across T servers. Any t of these servers can combine their signature shares to produce a valid aggregate signature for the message. At a high level, the three servers have respective public verification keys and secret signing keys. Each server generates a signature share for a message using its secret signing key if it considers the message abusive. Combining valid t signature shares results in an aggregate signature, which, along with the identifier tuple of the audited abusive message, is added to the blocklist. Such a method enables the verification by the user later on without learning which servers participate in the auditing for a message. Furthermore, no single server can forge an aggregate signature without access to at least t signing keys or signature shares. Here, we use a *2-out-of-3* instance where the EEMS, S_1, and S_2 independently audit the flagged messages. Each server generates a signature share for the identifier of a flagged message if it considers it abusive. They deliver the signature shares to each other, and each server can combine two signature shares for an identifier to produce an aggregate signature. Finally, each server holds the same blocklist that includes the identifiers with their aggregate signatures.

Oblivious detection. Now, the EEMS and $S_{\{1,2\}}$ hold identical blocklists $\mathcal{B}$ where each entry includes a message m's aggregate signature σ^* and its identifier tuple $\mathcal{I}$. Instead of encoding the entire message m being forwarded for detection purposes, a forwarder who possesses the related small-size $\mathcal{I}$ (as this tuple is attached to m when it reaches the recipient) can encode just $\mathcal{I}$. This encoded data can then be sent to the servers to determine whether a match on the blocklist exists, where the servers should

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

Algorithm 6: Transparent and Oblivious Detection (TODtc).

Input: The EEMS and $S_{\{1,2\}}$ hold $\ddot{\mathcal{I}}$, while the EEMS has key-value pairs where each pair is $\{\mathsf{H}(c) : (c, ik)\}$.
Ensure: The EEMS and $S_{\{1,2\}}$ hold an identical blocklist $\mathcal{B}$. The EEMS learns the sources of abusive messages.

$(sk_{EE}, pk_{EE}), (sk_{S_1}, pk_{S_1}), (sk_{S_2}, pk_{S_2}) \leftarrow \mathsf{TkeyGen}$; ▷ All public keys are known by all parties in the system
$pk_{dec} \leftarrow \mathsf{TkeyCom}(pk_{EE}, pk_{S_1}, pk_{S_2})$;
initiate the blocklist $\mathcal{B}$;
for $\mathcal{I} \in \ddot{\mathcal{I}}$ **do**
  get $mk, src, \mathsf{H}(c), \sigma$ from $\mathcal{I}$; $ch' \leftarrow \mathsf{E}_{mk}(\mathsf{H}(c))$;
  retrieve stored pair (c, ik); deliver c to $S_{\{1,2\}}$;
  Execute by the EEMS and $S_{\{1,2\}}$ independently: ▷ We use i to represent a single server
  if $1 \leftarrow \mathsf{Verify}(pk_E, \sigma, src, ch')$ **then** $m \leftarrow \mathsf{D}_{mk}(c)$;
    if m is audited as abusive **then**
      $\sigma_i \leftarrow \mathsf{TsignShare}(sk_i, m)$;
      i delivers the σ_i to the remaining two servers;
      a set A includes at least two signature shares;
      $\sigma^* \leftarrow \mathsf{Tcom}(A, m)$; $\mathcal{B}.\mathsf{add}(\{\mathcal{I} : \sigma^*\})$;
      $U_s \leftarrow \mathsf{D}_{ik}(src)$; ▷ Executed by EEMS
    end if
  end if
end for
A forwarder:
$(k_1, k_2) \leftarrow \mathsf{DPF.Gen}(\alpha = \tilde{\mathsf{H}}(\mathcal{I}), \beta = 1)$;
deliver k_1 and k_2 to random two servers;
Selected two servers:
$d_b = \sum_{j \in \mathcal{B}} \mathsf{DPF.Eval}(k_b, j.\tilde{\mathsf{H}}(\mathcal{I})) \cdot \mathsf{H}(j.\sigma^*)$ where $b = \{1, 2\}$;
send d_1 and d_2 to the forwarder;
if $d_1 + d_2 = \mathsf{H}(j.\sigma^*)$ where $j.\tilde{\mathsf{H}}(\mathcal{I}) = \alpha$ **then**
  the forwarded message is abusive;
end if

not infer the detected content and result. Within that, our problem can be formulated as private membership test. Although many works propose technical solutions that either require a trusted execution environment [45] (e.g., Intel SGX) or data structures (e.g., bloom filter [13], locality-sensitive hashing [12]) or two-party private set intersection protocols [46] (e.g., EMP), they suffer from side-channel attacks [47], false positives, or heavy interactions and costs [48], it is not available to integrate them into our system. Moreover, they fail to enable the verification of the forwarder on the detection result.

Our idea is inspired by oblivious systems based on DPF [49], [50], [51], [52], which is a very efficient and metadata-hiding way to solve this problem. Specifically, a forwarder can encode $\mathcal{I}$ as a key pair for a point function $f_{\alpha=\tilde{\mathsf{H}}(\mathcal{I}),\beta=1}$, where the key pair can be evaluated by two servers on the blocklist $\mathcal{B}$. The evaluation values return to the forwarder so as to reveal the detection result, where the forwarded message is abusive if the sum of the evaluation values is 1. However, this intuitive method still cannot allow the verification of the forwarder. We combine the aggregate signature with the evaluation process so as to enable the forwarder to obtain the hash of the corresponding signature only when a match occurs. If the forwarder does not trust the result, he/she can request the corresponding signature for verification. Besides, to add the trust of the forwarder, each detection randomly selects two servers from the EEMS and $S_{\{1,2\}}$ to execute the evaluations. From a technical standpoint, this application is quite simple. Yet, it promptly offers a more efficient, secure, and precise detection method.

$\mathsf{Game}^{\mathsf{EEMS}}_{\mathsf{System}}(\mathcal{A})$:
1) $\mathcal{A}$ is given access to some oracles and obtains some sessions and key-value pairs, i.e., $\mathsf{send}(m_i, \mathcal{I}_i; U_{s_i}, U_{r_i})$ and $\{\mathsf{H}(c_i), (c_i, ik_i)\}$, where $i \in$ a user set and no send-receive users (U_{s_i}, U_{r_i}) are controlled by $\mathcal{A}$.
2) The challenger chooses $b \leftarrow \{0, 1\}$:
   a) if $b = 0$, carries the SIdGen with the inputs from $\mathcal{A}$ and returns $\mathcal{A}$ the resulting EEMS view.
   b) if $b = 1$, chooses and sends random values with lengths $|c|$ and $|ch|$ to the EEMS, chooses random values with lengths $|m_i|$ and $|\mathcal{I}_i|$ and sends them from U_{s_i} to U_{r_i} using underlying send operation for each i.
3) $\mathcal{A}$ outputs a bit b' and has the advantage $\mathsf{Adv}^{\mathsf{EEMS}^1}_{\mathsf{System}}(\mathcal{A})$.
4) $\mathcal{A}$ is given access to some oracles and obtains key pairs and signature share, i.e., (k_1^j, k_2^j), σ_j, where $j \in$ a forwarder set.
5) The challenger chooses $b \leftarrow \{0, 1\}$:
   a) if $b = 0$, carries the TODtc with the inputs from $\mathcal{A}$ and returns $\mathcal{A}$ the resulting EEMS view.
   b) if $b = 1$, chooses and sends random values with lengths $|\sigma_j|$ and $|k_1^j|$ to the EEMS.
6) $\mathcal{A}$ outputs a bit b' and has the advantage $\mathsf{Adv}^{\mathsf{EEMS}^2}_{\mathsf{System}}(\mathcal{A})$.

Fig. 4. Security game against a semi-honest EEMS.

V. Security Analysis

We now show the formal security definitions for our proposed scheme and formally describe the guarantees of our required security properties under the game-based model.

We define indistinguishable games $\mathsf{Game}^{\mathcal{A}}_b$ where $b \in \{0, 1\}$ is a secret variable that decides the version of the game the probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ sees, and the advantage of the PPT $\mathcal{A}$ is defined as

$$\mathsf{Adv}^{\mathsf{game}}_{\mathsf{System}}(\mathcal{A}) = \left|\mathsf{Pr}\left[\mathsf{Game}^{\mathcal{A}}_1 = 1\right] - \mathsf{Pr}\left[\mathsf{Game}^{\mathcal{A}}_0 = 1\right]\right|.$$

We first show the security of the system against a semi-honest EEMS. In the first phase, the EEMS only learns who is communicating with whom and views random values that cannot reveal any information about messages. In the second phase, the EEMS generates input-independent correlation quantities, the key pairs for masked thresholds, and masks while receiving nothing in the running of the private report counting. At the end of the second phase, the EEMS receives identifier tuples from the two servers $S_{\{1,2\}}$. In the third phase, the EEMS is allowed to audit the corresponding messages, produce signature share for these messages, trace the corresponding sources, and learn that these messages have been reported more than the threshold t without knowing which reporters report which messages. When receiving encoded data from a forwarder, the EEMS should learn nothing else. We define the game $\mathsf{Game}^{\mathsf{EEMS}}_{\mathsf{System}}(\mathcal{A})$ between a PPT adversary $\mathcal{A}$ who controls the EEMS and some users and a challenger, as shown in Fig. 4.

Definition 1: We say that the scheme is secure against a semi-honest EEMS if the advantage of PPT $\mathcal{A}$ is negligible in $\mathsf{Game}^{\mathsf{EEMS}}_{\mathsf{System}}(\mathcal{A})$, i.e., $\mathsf{Adv}^{\mathsf{EEMS}^1}_{\mathsf{System}}(\mathcal{A}) + \mathsf{Adv}^{\mathsf{EEMS}^2}_{\mathsf{System}}(\mathcal{A}) \leq \mathsf{negl}(\lambda)$.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

Theorem 1: The system is secure against a semi-honest EEMS under Definition 1.

Proof: The EEMS can view c and ch on a call of SIdGen and the leakage from the underlying send and $\mathsf{receive}$. The challenger views the same leakage. c and ch are indistinguishable from random values since mk is randomly and uniformly selected and both $\mathsf{E}(\cdot)$ and $\mathsf{H}(\cdot)$ are computationally indistinguishable schemes, for any PPT $\mathcal{A}$, the probability of distinguish two ciphertexts is at most $1/2 + \mathsf{negl}(\lambda)$. The EEMS can generate key pairs for masked thresholds and masks on a call of $\mathsf{STrsEval}$, which are sent to $S_{\{1,2\}}$ for comparison. The EEMS cannot get anything from $S_{\{1,2\}}$ during the running of the second phase. Thus, the EEMS learns nothing in the second phase. The EEMS views an identifier matrix $\ddot{\mathcal{I}}$ at the end of running $\mathsf{STrsEval}$ so as to learn the message plaintext and message source while the view of all other messages is indistinguishable from uniformly random strings so that it cannot obtain anything about other messages. The EEMS also generates a signature share for the message that is deemed abusive. The EEMS cannot generate more signature shares so as to build an aggregate signature for a message due to the security guarantee of the threshold signature scheme.

We then show the security of our system against malicious users. A malicious source cannot send a message that cannot be traced to itself or send shares of a fake identifier tuple that passes the verification of the two servers. A malicious reporter (colluding with some reporters) cannot report a message except for the legal process. All users cannot learn the source of a received message in a forward state and report-related information. We define the game between a PPT adversary $\mathcal{A}$ who controls some malicious users and a challenger, as shown in Fig. 5. □

Definition 2: We say that the scheme is secure against malicious users if for any PPT $\mathcal{A}$ wins the above games with negligible probability, i.e., $\mathsf{Adv}^{\mathsf{U}_1}_{\mathsf{System}}(\mathcal{A}) + \mathsf{Adv}^{\mathsf{U}_2}_{\mathsf{System}}(\mathcal{A}) \leq \mathsf{negl}(\lambda)$.

Theorem 2: The system is secure against malicious users under Definition 2.

Proof: In the first phase, the view of $\mathcal{A}$ on a call of SIdGen involves pairs of src and σ. src and σ are encryption of the identity and signature generated from a secure encryption scheme $\mathsf{E}(\cdot)$ and a secure signature scheme $\mathsf{Sign}(\cdot)$, which have been proved that the outputs are indistinguishable for any PPT $\mathcal{A}$. mk is a uniformly random value and thus, any PPT $\mathcal{A}$ cannot distinguish between two uniformly random values. $\mathsf{H}(\cdot)$ is a random oracle. There is a negligible probability of outputting the same values with different inputs for all PPT $\mathcal{A}$. Hence, the probability of passing all verification on SIdVrf with a fake identifier tuple is negligible. In the second phase, the view of $\mathcal{A}$ on a call of $\mathsf{BPrfGen}$ involves pairs of γ and $([\tau]_1, [\tau]_2)$. $[\tau]_1$ and $[\tau]_2$ are secret shares generated from an ASS scheme, which have proved that the outputs are uniformly distributed in a ring $\mathbb{Z}_{2^{n+s}}$. γ is a uniformly random value and thus, any PPT $\mathcal{A}$ cannot distinguish between two uniformly random values. Hence, the probability of passing all verification on $\mathsf{BPrfVrf}$ with a fake identifier tuple or $\beta \neq 1$ is negligible. □

We finally show the security of our system against two semi-honest servers. We aim to argue that the two servers $S_{\{1,2\}}$ obtaining a shuffled identifier matrix $\boldsymbol{\mathcal{I}}^M$ cannot learn these messages and their sources. When a reporter sends a private report with proof, each server views random values without learning any message plaintext, source, and which message is reported by a reporter. After receiving a batch of private reports, each of them should not learn the explicit report count value of any message and the threshold beyond learning which messages have been reported more than a private threshold. When a forwarder delivers encoded data to S_1 or S_2, any of them should not learn the forwarded content or whether it matches any entry on the blocklist $\mathcal{B}$. Specifically, the two servers that are not colluding with each other learn from the running of the system. One of them may collude with some users. We define the following games between a PPT adversary $\mathcal{A}$ who controls a semi-honest server (e.g., S_1 or S_2) and some users and a challenger, as shown in Fig. 6.

$\mathsf{Game}^{\mathsf{U}}_{\mathsf{System}}(\mathcal{A})$:

1) $\mathcal{A}$ selects two messages m and m' with same length, who controls a user U_c. Honest users (U_a and U_b) are not controlled by $\mathcal{A}$.
2) The challenger chooses $b \leftarrow \{0,1\}$:
   a) if $b = 0$, carries the SIdGen with m from $\mathcal{A}$ and sends the m and identifier tuple $\mathcal{I}$ from U_a to U_b via the underlying send operation, carries the SIdGen with m from $\mathcal{A}$ and sends the m and identifier tuple from U_b to U_c via the underlying send operation.
   b) else, carries the SIdGen with m from $\mathcal{A}$ and sends the m and random values with length $|\mathcal{I}|$ from U_a to U_b via the underlying send operation, carries the SIdGen with m' from $\mathcal{A}$ and sends the m' and random values with length $|\mathcal{I}|$ from U_b to U_c via the underlying send operation, and returns $\mathcal{A}$ the resulting user view.
3) $\mathcal{A}$ outputs a bit b' and has the advantage $\mathsf{Adv}^{\mathsf{U}_1}_{\mathsf{System}}(\mathcal{A})$.
4) $\mathcal{A}$ calls SIdGen with the input m and obtains $\mathcal{I}$ with the source U_c. $\mathcal{A}$ generates m' and $\mathcal{I}'$ and sends them to U_a.
5) The challenger carries SIdVrf with the inputs m' and $\mathcal{I}'$.
6) $\mathcal{A}$ wins if SIdVrf successes.
7) The challenger calls secret-shared shuffle with the inputs from $\mathcal{A}$ and returns the resulting PMS view.
8) $\mathcal{A}$ wins if the inputs are not dropped.
9) $\mathcal{A}$ selects an identifier tuple $\mathcal{I}$.
10) The challenger chooses $b \leftarrow \{0,1\}$:
    a) if $b = 0$, carries the $\mathsf{BPrfGen}$ with $\mathcal{I}$ from $\mathcal{A}$ and sends the $\gamma, [\tau]_1, [\tau]_2$ to $S_{\{1,2\}}$, and returns $\mathcal{A}$ the resulting user view.
    b) else, carries the $\mathsf{BPrfGen}$ with $\mathcal{I}'$ from $\mathcal{A}$ and sends random values with lengths $|\gamma|, |[\tau]_1|, |[\tau]_2|$ to $S_{\{1,2\}}$.
11) $\mathcal{A}$ outputs a bit b' and has the advantage $\mathsf{Adv}^{\mathsf{U}_2}_{\mathsf{System}}(\mathcal{A})$.
12) $\mathcal{A}$ calls $\mathsf{BPrfGen}$ with the input $\mathcal{I}$ with the source U_c. $\mathcal{A}$ generates γ' and $([\tau]'_1, [\tau]'_2)$ and sends them to $S_{\{1,2\}}$.
13) The challenger carries $\mathsf{BPrfVrf}$ with the inputs γ' and $([\tau]'_1, [\tau]'_2)$.
14) $\mathcal{A}$ wins if $\mathsf{BPrfVrf}$ successes.

Fig. 5. Security game against malicious users.

Definition 3: We say that the scheme is secure against two semi-honest servers if the advantage of PPT $\mathcal{A}$ is negligible in $\mathsf{Game}^{\mathsf{PMS}}_{\mathsf{System}}(\mathcal{A})$, i.e., $\mathsf{Adv}^{\mathsf{PMS}_1}_{\mathsf{System}}(\mathcal{A}) + \mathsf{Adv}^{\mathsf{PMS}_2}_{\mathsf{System}}(\mathcal{A}) + \mathsf{Adv}^{\mathsf{PMS}_3}_{\mathsf{System}}(\mathcal{A}) \leq \mathsf{negl}(\lambda)$.

Theorem 3: The system is secure against two semi-honest servers under Definition 3.

Proof: Each server views shares and random values on a call of $\mathsf{RptTally}$ and $\mathsf{BPrfVrf}$. The challenger views the same values. The shares follow the FSS security guarantee, where each share reveals nothing else about the secret reported identifier tuple. The random value γ received from $\mathsf{BPrfVrf}$ is uniformly distributed in a ring and thus leaks nothing. The shares in $\mathsf{BPrfVrf}$ follow the underlying ASS security guarantee, where each share reveals nothing about the original secret. When $S_{\{1,2\}}$ execute $\mathsf{STrsEval}$, all values received from the EEMS are shares that have the

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

$\mathsf{Game}^{\mathsf{PMS}}_{\mathsf{System}}(\mathcal{A})$:

1) Once obtaining a shuffled identifier matrix, $\mathcal{A}$ views random value γ and shares $(k_1^i, [\tau]_1)$ from a reporter.
2) The challenger chooses $b \leftarrow \{0, 1\}$:
   a) if $b = 0$, carries the $\mathsf{CountRepo}$ with the inputs from $\mathcal{A}$ and returns $\mathcal{A}$ the resulting S_1 view.
   b) if $b = 1$, chooses and sends random values with length $|k_1^i|, |[\tau]_1|, |\gamma|$ to the S_1.
3) $\mathcal{A}$ outputs a bit b' and has the advantage $\mathsf{Adv}^{\mathsf{PMS}_1}_{\mathsf{System}}(\mathcal{A})$.
4) After obtaining report count shares of messages, $\mathcal{A}$ views κ_1, $[r]_1$, the report count shares $[y^x]_1$ of messages.
5) The challenger chooses $b \leftarrow \{0, 1\}$:
   a) if $b = 0$, carries the $\mathsf{STrsEval}$ with the inputs from $\mathcal{A}$ and returns $\mathcal{A}$ the resulting S_1 view.
   b) if $b = 1$, chooses and sends random values with lengths $|y^x + r|$ and $|\mathsf{DPF.Eval}(\kappa_1, y^x + r)|$ to the S_1, which locally computes the evaluation values and checks whether the sum of shares equals one for each message or not.
6) $\mathcal{A}$ outputs a bit b' and has the advantage $\mathsf{Adv}^{\mathsf{PMS}_2}_{\mathsf{System}}(\mathcal{A})$.
7) Within a blocklist $\mathcal{B}$, $\mathcal{A}$ views the encoded data k_1 from a forwarder.
8) The challenger chooses $b \leftarrow \{0, 1\}$:
   a) if $b = 0$, carries the TODtc with the inputs from $\mathcal{A}$ and returns $\mathcal{A}$ the resulting S_1 view.
   b) if $b = 1$, chooses and sends random values with lengths $|k_1|$ to the S_1, which locally computes the evaluation values and checks whether the sum of shares equals one for each message or not.
9) $\mathcal{A}$ outputs a bit b' and has the advantage $\mathsf{Adv}^{\mathsf{PMS}_3}_{\mathsf{System}}(\mathcal{A})$.

Fig. 6. Security game against two semi-honest servers.

same security level as the aforementioned algorithms. During the operation of obtaining masked report counts, the shares are masked by random values and thus are exactly indistinguishable. All values they view in the improved process of $\mathsf{DPF.Eval}$ are uniformly random by the definition of FSS and parity segment tree. Thus, all the inputs and outputs in TODtc are exactly indistinguishable from the values that would be sent and received in the real protocol. The underlying threshold signature scheme guarantees that a forged signature share cannot be verified and cannot be used to generate an aggregate signature that passes the verification. As a result, the two versions are indistinguishable for any PPT $\mathcal{A}$. □

VI. Implementation and Evaluation

Implementation. We provide a prototype implementation mainly in Rust, consisting of roughly 3.3 K LOC. We instantiate the symmetric encryption scheme with AES-128 GCM, the collision-resistant hash function with SHA-256, the cryptographic hash function with MD5, the signature scheme with ed25519, and the RSA threshold signature scheme [53]. We set the security parameters λ and s to 128 so that we sample shares and compute MACs using a ring $\mathbb{Z}_{2^{256}}$. We implement a distributed point function and distributed comparison function with SIMD and Hirose optimizations as libraries. Our optimized libraries are available as back-ends for other systems. Our system uses rocksdb[5] for the underlying key-value store. Additionally, we notice that the overall architecture is highly amenable to parallel processing, which enables us to utilize GPUs for simultaneous computations, leading to a significant enhancement in performance. We instantiate the GPU-based optimization with CUDA. The GPU implementation uses Salsa20 as the PRG for better performance [54]. Our implementation is open-source and available at https://github.com/myl7/nemesis/tree/plus.

[5]https://rocksdb.org/

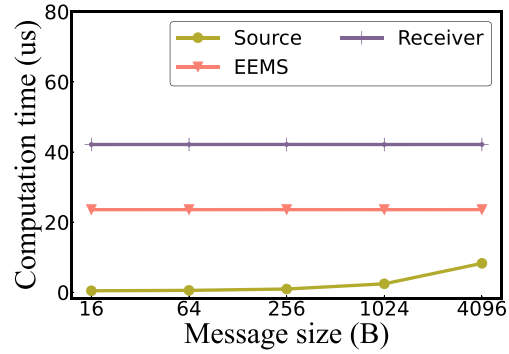


Fig. 7. Computation time of each entity in the phase of traceable message delivery with varying message sizes.

Experimental environment and parameters. We run the benchmarks on a machine running Ubuntu Linux with Intel i7-11700 K as the CPU and 64 GB of RAM. We establish 16 threads on 8 physical cores for evaluations. We run the GPU benchmarks on the same machine with NVIDIA RTX3080 as the GPU. We set the thread number per block to 512. Each experimental result is performed using Criterion and the average of 10 trials, with outliers removed. Message sizes in our experiments vary from 16 B to 4,096 B, following the settings in WhatsApp and Telegram. We explore different epoch settings, with the number M of messages per epoch ranging from 10^4 to 10^8. Following prior works [23], [24], the number N of reports per epoch ranges from 10^2 to 10^5.

A. Benchmarks: Costs of Each Entity

1) Traceable Message Delivery: We report our evaluation results for each entity in the phase of traceable message delivery with varying message sizes, as shown in Fig. 7. Overall, the communication and computation costs for users are low, requiring computation overheads at the microsecond level, which closely resemble the performance of an EEMS without reporting-then-tracing capabilities. Now, we detail the costs of each entity below.

Source's costs. The additional computation and communication costs of sending a message in our system are mainly from the identifier tuple generation and delivery. For the maximum message size 4,096 B, the computation time of generating the items of an identifier tuple is still fast and requires only $8.27\ \mu s$. The delivered identifier tuple size is independent of message size and only 164 B. To secret-share an identifier tuple to two servers, the only cost is to split it into two shares with length 164 B and the computation time is around $0.055\ \mu s$.

EEMS's costs. The EEMS is required to produce the items of an identifier tuple, i.e., the ciphertext of the source identity and a signature. The computation time is independent of message size and takes about $23.572 \mu s$. Subsequently, the EEMS performs a key-value append operation. We excluded the evaluation of this operation as it does not involve any cryptographic computations, and the time required for this operation is low in most industrial key-value stores. Baseline EEMS does not store message ciphertexts while reporting-then-tracing EEMSs [19], [20], [21], [22],

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPUTATION TIMES FOR OPERATIONS OF TWO SERVERS AND OFFLINE COMPUTATION TIME OF THE EEMS IN THE PHASE OF PRIVATE REPORT COUNTING WITH DIFFERENT NUMBERS OF MESSAGES

Number of messages	10^4	10^5	10^6	10^7
Two servers' costs				
Secret-shared shuffle (s)	0.034	0.339	3.37	33.9
Blind identifier verification (s)	0.003	0.032	0.290	2.969
Basic secure threshold evaluation (s)	0.031	0.31244	3.1349	31.345
Enhanced secure threshold evaluation (s)	0.024	0.242	2.430	24.158
EEMS's offline cost				
Generation for basic version (s)	0.069	0.703	6.9829	69.924
Generation for enhanced version (s)	0.056	0.566	5.616	56.334

TABLE II
A COMPARISON OF THE ADDITIONAL COSTS OF THE TREE-LINKABLE SCHEME OF PEALE ET AL. [20] AND OUR SYSTEM TO DELIVER AND REPORT A 1 KB MESSAGE

	Peale et al. [20]	Our system
Source	$101.79\mu s$ / 256 B	$2.4336\mu s$ / 164 B
Receiver	$105.67\mu s$ / 320 B	$42.143\mu s$ / 164 B
Reporter	$38.132\mu s$ / 160 B	$10.515\mu s$ / 2.957 KB

[23] require additional storage costs. In our system, the size of the stored key-value data per message is small and no more than 28 B. If the purpose is to address ongoing viral abusive messages, the EEMS can be designed to selectively store data from specific epochs within the current week or month to optimize storage costs.

Receiver's costs. The additional cost of a receiver is mainly from verifying the received identifier tuple. As message size increases from 16 B to 4,096 B, the computation time should increase as well since the running times of required symmetric encryption and hash operations are linear with the message size. We observe that the increased message sizes have a negligible impact on the overall time required. This is because the signature verification operation dominates the cost and remains independent of the message size. Based on our measurements, we report that the average time for this operation is approximately $42.143\mu s$.

2) Private Report Counting: As expected in our system, the computation times of most operations for two servers and the offline computation time for the EEMS grow with the number M of messages, as shown in Table I. The reporter spends constant computation costs to generate a report and associated proof. The results show that the costs are low even for handling reports over millions of messages.

Reporter's costs. To report a message, the costs for a reporter are composed of the DPF key generation and the proof generation. Both are independent of the reported message size. Based on our measurements, it takes around $10.287\mu s$ to generate a pair of DPF keys and around $0.228\mu s$ to generate the proof. Overall, the data sent to each server is 2.957 KB.

Two servers' costs. The total computation time to obtain a shuffled identifier matrix is low and is around $33.9s$ for 10^7 messages. Note that we do not include the preprocessing cost, where the EEMS can generate correlation quantities and deliver them to two servers during off-peak times. To blindly verify the proof for a report, the total computation time grows from 0.003 s to 2.969 s as an increase in M per epoch from 10^4 to 10^7. We measure the computation costs for both the basic and enhanced secure threshold evaluation methods. Overall, the enhanced version is over $1.3\times$ faster than the basic version. In particular, as M ranges from 10^4 to 10^7, the computation time for the basic version varies from 0.031 s to 31.345 s, while the computation time for the enhanced version varies from 0.024 s to 24.158 s. The computation time to update a report counter in an oblivious manner once receiving reports with proof from a set $\mathcal{N}$ of reporters increases with the number N of reports, which increases from 0.034 ms to 26.836 ms as N ranges from 10^2 to 10^5. Note that the above times are from a single server since the operations of the two servers can be done in parallel. While we do not measure the cost of network communication for this procedure, it is important to note that we do achieve it within our system, and we anticipate that it would not substantially increase costs. The communication cost of two servers is from returning the identifier tuples to the EEMS, which is linear with the number t' of messages whose report counts exceed the threshold and $164t'$ B.

EEMS's costs. The costs for the EEMS in the basic and enhanced secure threshold evaluations are mainly from generating masks and DCF/DPF keys and delivering them to the two servers, executed during the EEMS off-peak times. In brief, the enhanced version provides around $1.2\times$ speedup and reduces $1.77\times$ communication costs compared with the basic version. As M varies from 10^4 to 10^7, the offline computation costs range from 0.069 s to 69.924 s for the basic version and from 0.056 s to 56.334 s for the enhanced version. The total size sent to each server is linear with M, which is $5.143M$ KB for the basic version and $2.907M$ KB for the enhanced version.

3) Transparent and Oblivious Detection: To append an identifier tuple to the blocklist, each auditing participant creates a signature share and then computes an aggregate signature. The former process takes around 8.2 ms, while the latter requires around $0.252\mu s$. The size of a signature share is 674 B, and the size of an aggregate signature is 264 B. Verifying a signature share for a server requires around 6.49 ms while it takes about $50.644\mu s$ for a forwarder to verify an aggregate signature. Note that the verification costs exist when the received party does not trust the signature share or aggregate signature. The running time to trace a message source for the EEMS is constant and about $41.327\mu s$.

We also report the expenses for a forwarder to initiate detection and for the servers to execute detection across different numbers of identifier tuples included on the blocklist. A forwarder takes about $10.287\mu s$ to generate a DPF key pair, and each key is 2.895KB. Although the detection time of the servers increases with the growth of the number of identifier tuples on the blocklist, the time is pretty low for a server. In particular, as the growth of the number of identifier tuples on the blocklist increases from 10^2 to 10^5, the evaluation time for each server grows from $39.693\mu s$ to 32.638 ms.

B. Comparison With Prior Works

We compare our implementation with the open-source prototypes of Peale et al. [20] (tree-linkable version) that allows a single report to trigger tracing and Bell et al. [24] that achieves an exact threshold-based reporting. Both keep the same EEMS setting as us. We re-run their benchmarks on the same machine for fair comparisons. We compare with FACTS [23] at a high level due to its non-open-source nature.

Table II presents a comparison of the additional costs of the tree-linkable scheme of Peale et al. [20] and our system to deliver

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPUTATION COSTS OF DIFFERENT IMPLEMENTATIONS FOR TWO SERVERS AND THE EEMS WITH 10^8 MESSAGES

	CPU	GPU
Two servers' costs		
Secret-shared shuffle (s)	317.724	1.809
Blind identifier verification (s)	29.114	16.796
Enhanced secure threshold evaluation (s)	241.01	1.855
Oblivious detection (s)	29.114	16.579
EEMS's offline cost		
Generation for enhanced version (s)	561.69	39.069

and report a 1 KB message, both are independent of message size. The tree-linkable scheme of Peale et al. [20] incurs higher user computation overheads than our system. Meanwhile, the additional communication overheads of a source and receiver are also higher than the costs in ours. Although it requires a smaller communication overhead for reporting a message than our system, our system achieves report-hiding protection for a reporter and only requires a single flow of communication, which we think is affordable for both PC and mobile users. We observe that our system and the scheme of Bell et al. [24] use different methodologies for private exact threshold-triggered report counting. In their system, a server proves the report count of a message has triggered the threshold, and another server does the corresponding verification, resulting in the computation and communication complexity of $\mathcal{O}(N \cdot t)$, where N is the number of reports and t is the threshold. The computation costs for a server to prove and verify are around 28.4 ms and 31.4 ms when the threshold is 200. In contrast, in our system, the overhead of checking whether the report count of a message exceeds the threshold is constant and around $0.24\mu s$. As the scheme of Bell et al. [24] does not implement the send and receive operations, it is hard to compare the additional overheads between their scheme and ours. Here, we compare the computation and communication overheads for a reporter between their scheme and ours. The report time of their scheme is higher than ours, which requires about $360\mu s$ for a user to report a message, while the report communication cost is smaller than ours and is 176 B.

For our system and FACTS [23], there are differences in terms of: (1) *security:* our system protects the report count of any message and the threshold against users, while reporters in FACTS can learn this information; (2) *accuracy of report counting:* our system ensures exact report tally while they provide approximate report tally; (3) *performance:* our system allows issuing a report with low running time (around $10.515\mu s$), while the time of issuing a report in FACTS is higher for lower threshold, requiring around 375 ms for a reporter when the threshold is 200.

C. GPU-Based Optimization

We optimize several operations for two servers, including the secret-shared shuffle, private identifier verification, and enhanced secure threshold evaluation. Additionally, we have improved the generation process for the enhanced version on the EEMS side. Table III shows the comparison in the computation costs of our basic CPU-based implementation and optimized GPU-based implementation over 10^8 messages. It is evident that the GPU-based parallel implementation reduces the running time by about $2 - 170\times$ speedup. It is noted that private identifier verification and oblivious detection need to sum up the 10^8 results, which cannot be highly parallelized. Thus, we do the sum operation on the CPU while still optimizing DPF evaluations with GPU.

VII. RELATED WORK

Existing methods in the literature are mainly built on user-reporting-based and blocklist-based methods[6].

The first line of approach is to leverage messaging users to report potentially abusive messages to EEMS. Message franking [15], [16], [17], [18], [25] is an initial cryptographic tool that enables a reporter to verifiably report a message to EEMS, which reveals the associated message sender. The following research focuses on source traceback techniques, which enable EEMS to identify the source of a reported message. Most existing works on source traceback [19], [20], [21], [22] consider a single report to trigger tracing. FACTS system [23] supports approximate threshold report counting before source tracing, while the work [24] allows exact threshold-based report tally. However, they fail to support flexible and adaptable thresholds for satisfying a practical need in content moderation for EEMS. In contrast, our system enables exact threshold-based report counting where the EEMS can flexibly set the threshold without needing a system reset or additional costs. Besides, apart from disclosing the source identity of an abusive message, prior works enable EEMS to gain knowledge of either the complete or partial identity of intermediate users associated with the reported message, thereby exposing the connection between users and reported messages. Our system guarantees beyond the source of an abusive message, all other users related to the reported message remain undisclosed.

Another line of way to moderate encrypted content is to proactively scan the ciphertext and check whether a match with a pre-defined blocklist (e.g., hash values of abusive contents) exists [10], [11], [12], [13], [14]. An EEMS possessing the blocklist runs a private set intersection protocol with the user to check whether a match exists or not. The blocklist used in these works is assumed to be trustworthy, while a user cannot verify it. In contrast to these studies, our system establishes a transparent blocklist, where the auditing of each entry is finished by at least two servers, and allows users to confirm the consensus on the auditing of messages. Moreover, they focus on the image scenario and thus suffer from false positives due to the similarity testing designs, while our system supports exact detection for identified abusive content. The work [56] proposes an enhanced version of Apple PSI [57] that allows multiple child safety organizations to create threshold signatures for the hash values of child sexual abuse material (CSAM) items on a hash set, where a user can verify the threshold intersection. However, the enhanced version of Apple PSI in the work [56] cannot be directly used to achieve the desired transparent and oblivious detection service in our system. The work in [56] and our work consider different application scenarios, which lead to different design requirements. Their work only enables Apple to obtain the hash set with aggregate signatures, while our work enables all servers to obtain the same blocklist with aggregate signatures. In particular, their work requires the servers (i.e., child safety organizations) to generate blinded signature shares for the hash values of abusive content without learning the blinded key of Apple, where Apple generates aggregate signatures for the hash

[6]We refer readers to [55] for more detailed information about content moderation in EEMSs.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

values of abusive content that at least t-out-of-T servers contribute blinded signature shares. Our work allows these servers to produce signature shares for the hash values of identifiers directly, where each server generates aggregate signatures for the hash values of identifiers that at least t-out-of-T servers contribute signature shares. Besides, they cannot guarantee that all items contained in the hash set are truly abusive content, since these organizations have to sign some dummy items for security guarantees, while ours does not sign such dummy items.

Differences from conference version. Portions of this work were presented in [1]. This extended version includes significant revisions and new contributions summarized as follows: First, we propose a new protocol in Section IV-C for proactive abusive content detection in EEMSs, enhancing functionality while preserving message confidentiality and detection efficiency. Second, we introduce a new secure threshold evaluation method in Section IV-B3, optimizing computational and communication costs for large-scale message volumes and improving scalability. Third, we formalize security guarantees in Section V with game-based definitions and proofs, covering new detection and secure threshold evaluation methods. Fourth, Section VI provides extensive performance evaluations, including new protocols, comparison with prior works, and GPU-accelerated instantiations, demonstrating substantial improvements. Finally, the paper has been thoroughly revised, including updates to the abstract, introduction, system model, detailed design, and related work.

VIII. Conclusion

We propose a new privacy-aware moderation system for combating abusive information in EEMSs through tailored lightweight cryptographic protocols. We first propose a privacy-preserving design for counting reports, threshold evaluation, and tracing back the sources of abusive messages. We then further propose a design for building a transparent blocklist to privately detect forwarders' messages, so as to mitigate the further spread of identified abusive messages from the user-reporting-based process. Extensive experiments demonstrate the promising performance of our system.

References

[1] R. Lian, Y. Ming, C. Cai, Y. Zheng, C. Wang, and X. Jia, "Nemesis: Combating abusive information in encrypted messaging with private reporting," in *Proc. 29th Eur. Symp. Res. Comput. Secur.*, 2024, pp. 247–267.

[2] S. Dechand, A. Naiakshina, A. Danilova, and M. Smith, "In encryption we don't trust: The effect of end-to-end encryption to the masses on user perception," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2019, pp. 401–415.

[3] M. Isaac and K. Roose, "Disinformation spreads on Whatsapp ahead of Brazilian election," 2018. [Online]. Available: https://www.nytimes.com/2018/10/19/technology/whatsapp-brazil-presidential-election.html

[4] CCDH, "One of the biggest publishers of election misinfo earned up to $1.1 million in google ad revenue," 2018. [Online]. Available: https://counterhate.com/blog/one-of-the-biggest-publishers-of-election-misinfo-earned-up-to-1-1-million-in-google-ad-revenue/

[5] L. Belli, "Whatsapp skewed Brazilian election, showing social media's danger to democracy," 2018. [Online]. Available: https://theconversation.com/whatsapp-skewed-brazilian-election-showing-social-medias-danger-to-democracy-106476

[6] E. Dwoskin and A. Gowen, "On Whatsapp, fake news is fast — and can be fatal," 2018. [Online]. Available: https://www.washingtonpost.com/business/economy/on-whatsapp-fake-news-is-fast-and-can-be-fatal/

[7] K. Starbird, *Online Rumors, Misinformation and Disinformation: The Perfect Storm of Covid-19 and Election2020*. Berkeley, CA, USA: USENIX Association, 2021.

[8] P. P et al., "Open letter: Facebook's "privacy first" proposals," 2018. [Online]. Available: https://www.justice.gov/opa/press-release/file/1207081/download

[9] P. P. et al., "International statement: End-to-end encryption and public safety," 2020. [Online]. Available: https://www.justice.gov/opa/pr/international-statement-end-end-encryption-and-public-safety

[10] Apple, "Csam detection," 2023. [Online]. Available: https://www.apple.com/child-safety/pdf/CSAM_Detection_Technical_Summary.pdf

[11] A. Kulshrestha and J. Mayer, "Identifying harmful media in {End-to-End} encrypted communication: Efficient private membership computation," in *Proc. USENIX Secur. Symp.*, 2021, pp. 893–910.

[12] Y. Hua, A. Namavari, K. Cheng, M. Naaman, and T. Ristenpart, "Increasing adversarial uncertainty to scale private similarity testing," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1777–1794.

[13] T. Zheng, T. Zhou, K. Lu, and Z. Cai, "Inspecting end-to-end encrypted communication differentially for the efficient identification of harmful media," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 5772–5787, 2023.

[14] J. Bartusek, S. Garg, A. Jain, and G.-V. Policharla, "End-to-end secure messaging with traceability only for illegal content," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2023, pp. 35–66.

[15] P. Grubbs, J. Lu, and T. Ristenpart, "Message franking via committing authenticated encryption," in *Proc. Annu. Int. Cryptology Conf.*, 2017, pp. 66–97.

[16] Y. Dodis, P. Grubbs, T. Ristenpart, and J. Woodage, "Fast message franking: From invisible Salamanders to encryptment," in *Proc. Annu. Int. Cryptology Conf.*, 2018, pp. 155–186.

[17] N. Tyagi, P. Grubbs, J. Len, I. Miers, and T. Ristenpart, "Asymmetric message franking: Content moderation for metadata-private end-to-end encryption," in *Proc. Annu. Int. Cryptology Conf.*, 2019, pp. 222–250.

[18] M. Gregoire, M. Pierce, and S. Eskandarian, "Onion Franking: Abuse reports for mix-based private messaging," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2025.

[19] N. Tyagi, I. Miers, and T. Ristenpart, "Traceback for end-to-end encrypted messaging," in *Proc. 2019 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 413–430.

[20] C. Peale, S. Eskandarian, and D. Boneh, "Secure complaint-enabled source-tracking for encrypted messaging," in *Proc. 2021 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 1484–1506.

[21] E. Kenney, Q. Tang, and C. Wu, "Anonymous traceback for end-to-end encryption," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2022, pp. 42–62.

[22] R. Issa, N. Alhaddad, and M. Varia, "Hecate: Abuse reporting in secure messengers with sealed sender," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 2335–2352.

[23] L. Liu, D. S. Roche, A. Theriault, and A. Yerukhimovich, "Fighting fake news in encrypted messaging with the fuzzy anonymous complaint tally system (FACTS)," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2022.

[24] C. Bell and S. Eskandarian, "Anonymous complaint aggregation for secure messaging," in *Proc. Annu. Privacy Enhancing Technol. Symp.*, 2024, pp. 276–296.

[25] S. Eskandarian, "Abuse reporting for {Metadata-Hiding} communication based on secret sharing," in *Proc. USENIX Secur.*, 2024, pp. 3205–3221.

[26] M. Shtaya, "How meta's platforms normalize anti-palestinian racism," 2023. [Online]. Available: https://mei.edu/publications/how-metas-platforms-normalize-anti-palestinian-racism

[27] E. Boyle, N. Gilboa, and Y. Ishai, "Function secret sharing," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2015, pp. 337–367.

[28] S. Eskandarian and D. Boneh, "Clarion: Anonymous communication from multiparty shuffling protocols," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2022.

[29] K. Storrier, A. Vadapalli, A. Lyons, and R. Henry, "Grotto: Screaming fast (2 1)-PC or z_{2^n} via (2, 2)-DPFS," in *Proc. 2023 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 2143–2157.

[30] J. F. Almansa, I. Damgård, and J. B. Nielsen, "Simplified threshold RSA with adaptive and proactive security," in *Proc. 24th Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2006, pp. 593–611.

[31] B. Kaiser, J. Wei, E. Lucherini, K. Lee, J. N. Matias, and J. Mayer, "Adapting security warnings to counter online disinformation," in *Proc. USENIX Secur.*, 2021, pp. 1163–1180.

[32] P. Rösler, C. Mainka, and J. Schwenk, "More is less: On the end-to-end security of group chats in signal, Whatsapp, and threema," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2018, pp. 415–429.

[33] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, "A formal security analysis of the signal messaging protocol," *J. Cryptol.*, vol. 33, pp. 1914–1983, 2020.

[34] K. Zhong, Y. Ma, and S. Angel, "Ibex: Privacy-preserving Ad conversion tracking and bidding," in *Proc. 2022 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 3223–3237.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.

[35] W. Chen and R. A. Popa, "Metal: A metadata-hiding file-sharing system," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020.

[36] Z. Newman, S. Servan-Schreiber, and S. Devadas, "Spectrum: High-bandwidth anonymous broadcast," in *Proc. 19th USENIX Symp. Netw. Syst. Des. Implementation*, 2022.

[37] N. Gilboa and Y. Ishai, "Distributed point functions and their applications," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2014, pp. 640–658.

[38] R. Cramer, I. Damgård, D. Escudero, P. Scholl, and C. Xing, "Spd$\mathbb{Z}_{2^k}$: Efficient mpc mod 2^k for dishonest majority," in *Proc. Annu. Int. Cryptology Conf.*, 2018, pp. 769–798.

[39] F. Benhamouda, A. Degwekar, Y. Ishai, and T. Rabin, "On the local leakage resilience of linear secret sharing schemes," *J. Cryptol.*, vol. 34, pp. 1–65, 2021.

[40] E. Boyle, N. Gilboa, and Y. Ishai, "Function secret sharing: Improvements and extensions," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 1292–1303.

[41] D. Boneh, E. Boyle, H. Corrigan-Gibbs, N. Gilboa, and Y. Ishai, "Lightweight techniques for private heavy hitters," in *Proc. IEEE Symp. Secur. Privacy*, 2021, pp. 762–776.

[42] D. Beaver, "Efficient multiparty protocols using circuit randomization," in *Proc. Annu. Int. Cryptology Conf.*, 1992, pp. 420–432.

[43] E. Boyle, N. Gilboa, and Y. Ishai, "Secure computation with preprocessing via function secret sharing," in *Proc. Theory Cryptography Conf.*, 2019, pp. 341–371.

[44] E. Boyle et al., "Function secret sharing for mixed-mode and fixed-point secure computation," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2021, pp. 871–900.

[45] S. Tamrakar, J. Liu, A. Paverd, J.-E. Ekberg, B. Pinkas, and N. Asokan, "The circle game: Scalable private membership test using trusted hardware," in *Proc. 2017 ACM Asia Conf. Comput. Commun. Secur.*, 2017, pp. 31–44.

[46] B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "SpOT-light: Lightweight private set intersection from sparse OT extension," in *Proc. Annu. Int. Cryptology Conf.*, 2019, pp. 401–431.

[47] K. Murdock, D. Oswald, F. D. Garcia, J. Van Bulck, D. Gruss, and F. Piessens, "Plundervolt: Software-based fault injection attacks against intel SGX," in *Proc. IEEE Symp. Secur. Privacy*, 2020, pp. 1466–1482.

[48] Y. Jia, S.-F. Sun, H.-S. Zhou, J. Du, and D. Gu, "Shuffle-based private set union: Faster and more secure," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 2947–2964.

[49] H. Corrigan-Gibbs, D. Boneh, and D. Mazières, "Riposte: An anonymous messaging system handling millions of users," in *Proc. IEEE Symp. Secur. Privacy*, 2015, pp. 321–338.

[50] S. Eskandarian, H. Corrigan-Gibbs, M. Zaharia, and D. Boneh, "Express: Lowering the cost of metadata-hiding communication with cryptographic privacy," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 1775–1792.

[51] D. Demmler, P. Rindal, M. Rosulek, and N. Trieu, "PIR-PSI: Scaling private contact discovery," *Proc. Priv. Enhancing Technol.*, vol. 2018, no. 4, pp. 159–178, 2018.

[52] F. Wang, C. Yun, S. Goldwasser, V. Vaikuntanathan, and M. Zaharia, "Splinter: Practical private queries on public data," in *Proc. 14th USENIX Symp. Netw. Syst. Des. Implementation*, 2017, pp. 299–313.

[53] V. Shoup, "Practical threshold signatures," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, 2000, pp. 207–220.

[54] M. Lam et al., "GPU-based private information retrieval for on-device machine learning inference," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*2024, pp. 197–214. [Online]. Available: https://dl.acm.org/doi/10.1145/3617232.3624855

[55] C. Rahalkar and A. Virgaonkar, "SoK: Content moderation schemes in end-to-end encrypted systems," 2022, *arXiv:2208.11147*.

[56] S. Scheffler, A. Kulshrestha, and J. Mayer, "Public verification for private hash matching," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 253–273.

[57] A. Bhowmick, D. Boneh, S. Myers, K. Talwar, and K. Tarbe, "The apple psi system," 2023. [Online]. Available: https://www.apple.com/child-safety/pdf/Apple_PSI_System_Security_Protocol_and_Analysis.pdf

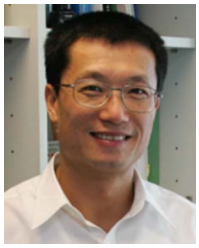
Rui Lian is currently working toward the PhD degree with the Department of Computer Science, City University of Hong Kong. Her research interests include building secure decentralized applications with the help of trusted execution environments and cryptography.

Yifeng Zheng (Member, IEEE) is an assistant professor with the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University. His research works have been published in prestigious venues like USENIX Security, CCS, VLDB, TIFS, and TDSC. He has served as an associate editor for *IEEE Transactions on Services Computing*. He has served on the technical program committee of many major international conferences, such as EuroS&P'25, PETS'25, AsiaCCS'25, PAKDD'25, CIKM'24, WISE'24, and ICDCS'24. He received the Best Paper Award in the European Symposium on Research in Computer Security (ESORICS) 2021. His current research interests are focused on machine learning security and privacy, secure networked systems, and secure outsourced computation.

Yulong Ming is currently working toward the PhD degree with the Department of Computer Science, City University of Hong Kong. His research interests include network security and cryptography.

Chengjun Cai (Member, IEEE) is currently an associate professor with City University of Hong Kong (Dongguan). His research interests included applied cryptography, data security and privacy, and blockchain. He is a member of the ACM.

Cong Wang (Fellow, IEEE) is a professor in the Department of Computer Science, City University of Hong Kong. His research interests include data and network security, blockchain and decentralized applications, and privacy-enhancing technologies. He has been one of the Founding Members of the Young Academy of Sciences of Hong Kong since 2017, and has been conferred the RGC Research Fellow in 2021. He received the Outstanding Researcher Award (junior faculty) in 2019, the Outstanding Supervisor Award, in 2017 and the president's Awards, in 2019 and 2016, all from City University of Hong Kong. He is a co-recipient of the Best Paper Award of IEEE ICDCS 2020, ICPADS 2018, MSN 2015, the Best Student Paper Award of IEEE ICDCS 2017, and the IEEE INFOCOM Test of Time Paper Award 2020. His research has been supported by multiple government research fund agencies, including National Natural Science Foundation of China, Hong Kong Research Grants Council, and Hong Kong Innovation and Technology Commission. He has served as editor-in-chief for *IEEE Transactions on Dependable and Secure Computing (TDSC)* and associate editor for *IEEE Transactions on Services Computing (TSC)*, *IEEE Internet of Things Journal (IoT-J)*, *IEEE Networking Letters*, and *Journal of Blockchain Research*, and TPC co-chairs for a number of IEEE conferences and workshops.



Xiaohua Jia (Fellow, IEEE) received the BSc and MEng degrees(1987) from the University of Science and Technology of China, in 1984 and 1987 respectively, and the DSc degree in information science from the University of Tokyo, in 1991. He is currently chair professor with Dept. of Computer Science with the City University of Hong Kong. His research interests include cloud computing and distributed systems, computer networks and mobile computing. Prof. Jia is an editor of *IEEE Internet of Things*, *IEEE Transactions on Parallel and Distributed Systems (2006-2009)*, *Wireless Networks*, *Journal of World Wide Web*, *Journal of Combinatorial Optimization*, etc. He is the general chair of ACM MobiHoc 2008, TPC Co Chair of IEEE GlobeCom 2010 Ad Hoc and Sensor Networking Symposium, Area-Chair of IEEE INFOCOM 2010 and 2015.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:25:01 UTC from IEEE Xplore. Restrictions apply.